Async Tests

You have already seen how to test your **FastAPI** applications using the provided `TestClient`. Up to now, you have only seen how to write synchronous tests, without using `async` functions.

Being able to use asynchronous functions in your tests could be useful, for example, when you're querying your database asynchronously. Imagine you want to test sending requests to your FastAPI application and then verify that your backend successfully wrote the correct data in the database, while using an async database library.

Let's look at how we can make that work.

pytest.mark.anyio

If we want to call asynchronous functions in our tests, our test functions have to be asynchronous. AnyIO provides a neat plugin for this, that allows us to specify that some test functions are to be called asynchronously.

HTTPX

Even if your **FastAPI** application uses normal `def` functions instead of `async def`, it is still an `async` application underneath.

The `TestClient` does some magic inside to call the asynchronous FastAPI application in your normal `def` test functions, using standard pytest. But that magic doesn't work anymore when we're using it inside asynchronous functions. By running our tests asynchronously, we can no longer use the `TestClient` inside our test functions.

The `TestClient` is based on HTTPX, and luckily, we can use it directly to test the API.

Example

For a simple example, let's consider a file structure similar to the one described in Bigger Applications and Testing:

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   └── test_main.py
```

The file `main.py` would have:

Python 3.10+

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
async def root():
    return {"message": "Tomato"}
```

The file `test_main.py` would have the tests for `main.py`, it could look like this now:

Python 3.10+

```python
import pytest
from httpx import ASGITransport, AsyncClient

from .main import app


@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url="http://test"
    ) as ac:
        response = await ac.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Tomato"}
```

Run it

You can run your tests as usual via:

```bash
```

In Detail

The marker `@pytest.mark.anyio` tells pytest that this test function should be called asynchronously:

Python 3.10+

```python
import pytest
from httpx import ASGITransport, AsyncClient

from .main import app


@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url="http://test"
    ) as ac:
        response = await ac.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Tomato"}
```

Tip

Note that the test function is now `async def` instead of just `def` as before when using the `TestClient`.

Then we can create an `AsyncClient` with the app, and send async requests to it, using `await`.

Python 3.10+

```python
import pytest
from httpx import ASGITransport, AsyncClient

from .main import app


@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url="http://test"
    ) as ac:
        response = await ac.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Tomato"}
```

This is the equivalent to:

```python
response = client.get('/')
```

...that we used to make our requests with the `TestClient`.

Tip

Note that we're using async/await with the new `AsyncClient` - the request is asynchronous.

Warning

If your application relies on lifespan events, the `AsyncClient` won't trigger these events. To ensure they are triggered, use `LifespanManager` from florimondmanca/asgi-lifespan.

Other Asynchronous Function Calls

As the testing function is now asynchronous, you can now also call (and `await`) other `async` functions apart from sending requests to your FastAPI application in your tests, exactly as you would call them anywhere else in your code.

Tip

If you encounter a `RuntimeError: Task attached to a different loop` when integrating asynchronous function calls in your tests (e.g. when using MongoDB's MotorClient), remember to instantiate objects that need an event loop only within async functions, e.g. an `@app.on_event("startup")` callback.